

**IMPLEMENTATION OF LOGIC GATES  
USING ONLY NAND GATES**

**NAND2Tetris HDL Assignment**

|                      |                                    |
|----------------------|------------------------------------|
| <b>Name</b>          | Divyansh Sharma                    |
| <b>Roll No.</b>      | 2501730385                         |
| <b>Course</b>        | B.Tech CSE (AI & ML)               |
| <b>Section</b>       | E                                  |
| <b>Software Used</b> | NAND2Tetris Web Hardware Simulator |

**Objective**

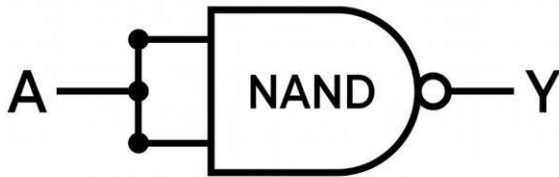
- **To implement NOT, AND, and OR logic gates using only NAND gates in HDL, verify their operation with the provided NAND2Tetris test scripts, and document the corresponding logic diagrams and simulation evidence.**

## 1. Logic Diagrams

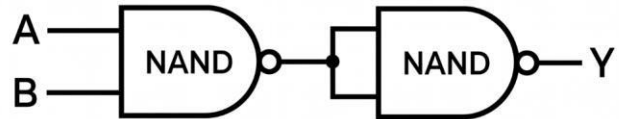
The following diagram shows the NAND-only implementations of the NOT, AND, and OR gates. The diagrams use standard NAND symbols and clearly show how the inputs are connected.

### IMPLEMENTATION OF LOGIC GATES USING ONLY NAND GATES

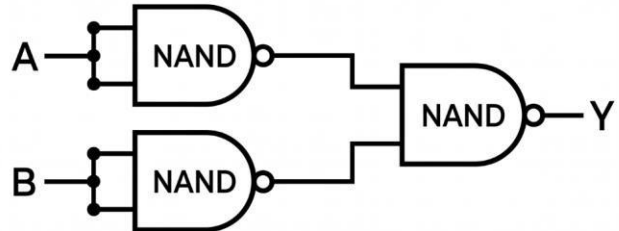
#### SECTION 1 — NOT GATE USING NAND



#### SECTION 2 — AND GATE USING NAND



#### SECTION 3 — OR GATE USING NAND



## 2. NOT Gate Using NAND

A NOT gate can be implemented using one NAND gate by connecting the same input to both NAND inputs.

### 2.1 Boolean Expression

$$Y = (A \cdot A)' = A'$$

### 2.2 HDL Code

```
CHIP Not {  
    IN in;  
    OUT out;  
  
    PARTS:  
        Nand(a=in, b=in, out=out);  
}
```

### 2.3 Truth Table

| A (in) | Y (out) |
|--------|---------|
| 0      | 1       |
| 1      | 0       |

### 2.4 Test Script: Not.tst

```
// This file is part of www.nand2tetris.org  
// and the book "The Elements of Computing Systems"  
// by Nisan and Schocken, MIT Press.  
// File name: projects/1/Not.tst  
  
load Not.hdl,  
compare-to Not.cmp,  
output-list in out;  
  
set in 0,  
eval,  
output;  
  
set in 1,  
eval,  
output;
```

### 2.5 Expected Compare Output

| in | out |
|----|-----|
| 0  | 1   |
| 1  | 0   |

### 3. AND Gate Using NAND

An AND gate can be implemented with two NAND gates. The first NAND produces the complemented AND result, and the second NAND acts as an inverter by receiving the same intermediate signal on both inputs.

#### 3.1 Boolean Expression

$$X = (A \cdot B)' \quad \text{and} \quad Y = (X \cdot X)' = A \cdot B$$

#### 3.2 HDL Code

```
CHIP And {  
  IN a, b;  
  OUT out;  
  
  PARTS:  
    Nand(a=a, b=b, out=nandOut);  
    Nand(a=nandOut, b=nandOut, out=out);  
}
```

#### 3.3 Truth Table

| A (a) | B (b) | Y (out) |
|-------|-------|---------|
| 0     | 0     | 0       |
| 0     | 1     | 0       |
| 1     | 0     | 0       |
| 1     | 1     | 1       |

#### 3.4 Test Script: And.tst

```
// This file is part of www.nand2tetris.org  
// and the book "The Elements of Computing Systems"  
// by Nisan and Schocken, MIT Press.  
// File name: projects/1/And.tst  
  
load And.hdl,  
compare-to And.cmp,  
output-list a b out;  
  
set a 0,  
set b 0,  
eval,  
output;  
  
set a 0,
```

```

set b 1,
eval,
output;

set a 1,
set b 0,
eval,
output;

set a 1,
set b 1,
eval,
output;

```

### 3.6 Expected Compare Output

| a | b | out |
|---|---|-----|
| 0 | 0 | 0   |
| 0 | 1 | 0   |
| 1 | 0 | 0   |
| 1 | 1 | 1   |

## 4. OR Gate Using NAND

An OR gate can be implemented using three NAND gates. The first two NAND gates invert A and B, and the third NAND combines the inverted signals.

### 4.1 Boolean Expression

$$Y = (A' \cdot B')' = A + B$$

### 4.2 HDL Code

```

CHIP Or {
  IN a, b;
  OUT out;

  PARTS:
    Nand(a=a, b=a, out=notA);
    Nand(a=b, b=b, out=notB);
    Nand(a=notA, b=notB, out=out);
}

```

### 4.3 Truth Table

| A (a) | B (b) | Y (out) |
|-------|-------|---------|
| 0     | 0     | 0       |
| 0     | 1     | 1       |
| 1     | 0     | 1       |
| 1     | 1     | 1       |

### 4.4 Test Script: Or.tst

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Or.tst
```

```
load Or.hdl,
compare-to Or.cmp,
output-list a b out;
```

```
set a 0,
set b 0,
eval,
output;
```

```
set a 0,
set b 1,
eval,
output;
```

```
set a 1,
set b 0,
eval,
output;
```

```
set a 1,
set b 1,
eval,
output;
```

#### 4.5 Expected Compare Output

| <b>a</b> | <b>b</b> | <b>out</b> |
|----------|----------|------------|
| 0        | 0        | 0          |
| 0        | 1        | 1          |
| 1        | 0        | 1          |
| 1        | 1        | 1          |

#### 5. Summary

| <b>Gate</b> | <b>NAND Gates Used</b> | <b>Equivalent Boolean Operation</b> |
|-------------|------------------------|-------------------------------------|
| NOT         | 1                      | $Y = A'$                            |
| AND         | 2                      | $Y = A \cdot B$                     |
| OR          | 3                      | $Y = A + B$                         |

#### 6. Test Files Included

The provided NAND2Tetris test scripts are included in this document for reproducibility:

- Not.tst
- And.tst
- Or.tst

#### 7. Conclusion

NOT, AND, and OR gates were implemented using only NAND gates in HDL. The supplied NAND2Tetris test scripts define the required input combinations and compare the generated results against the expected compare files. The AND-gate simulator screenshot is included as execution evidence, while the expected outputs for all three gates are documented from their corresponding compare files.

## 8. Simulation Evidence

The following screenshots provide evidence of the NOT, AND, and OR gate HDL implementations and their corresponding NAND2Tetris test environments.

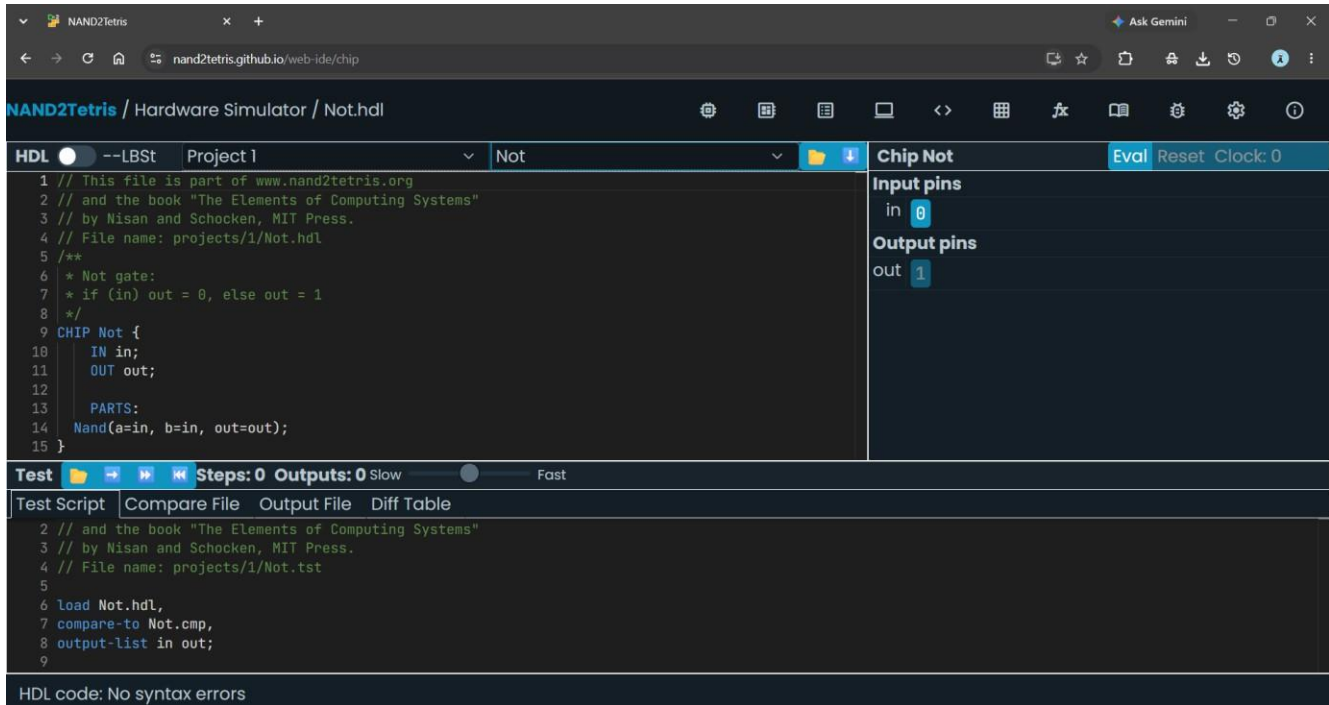


Figure 3. NOT gate HDL implementation, Chip/Pins panel, and Not.tst test panel.



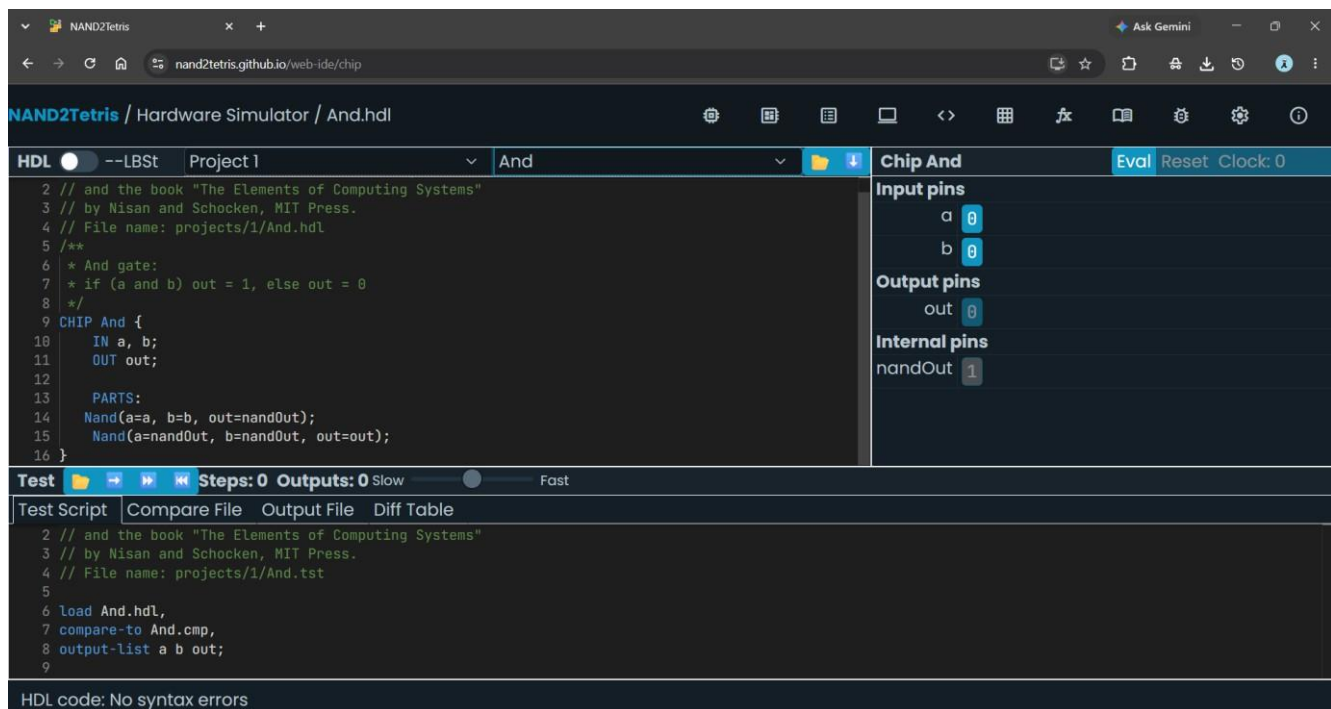


Figure 4. AND gate HDL implementation, Chip/Pins panel, and And.tst test panel.

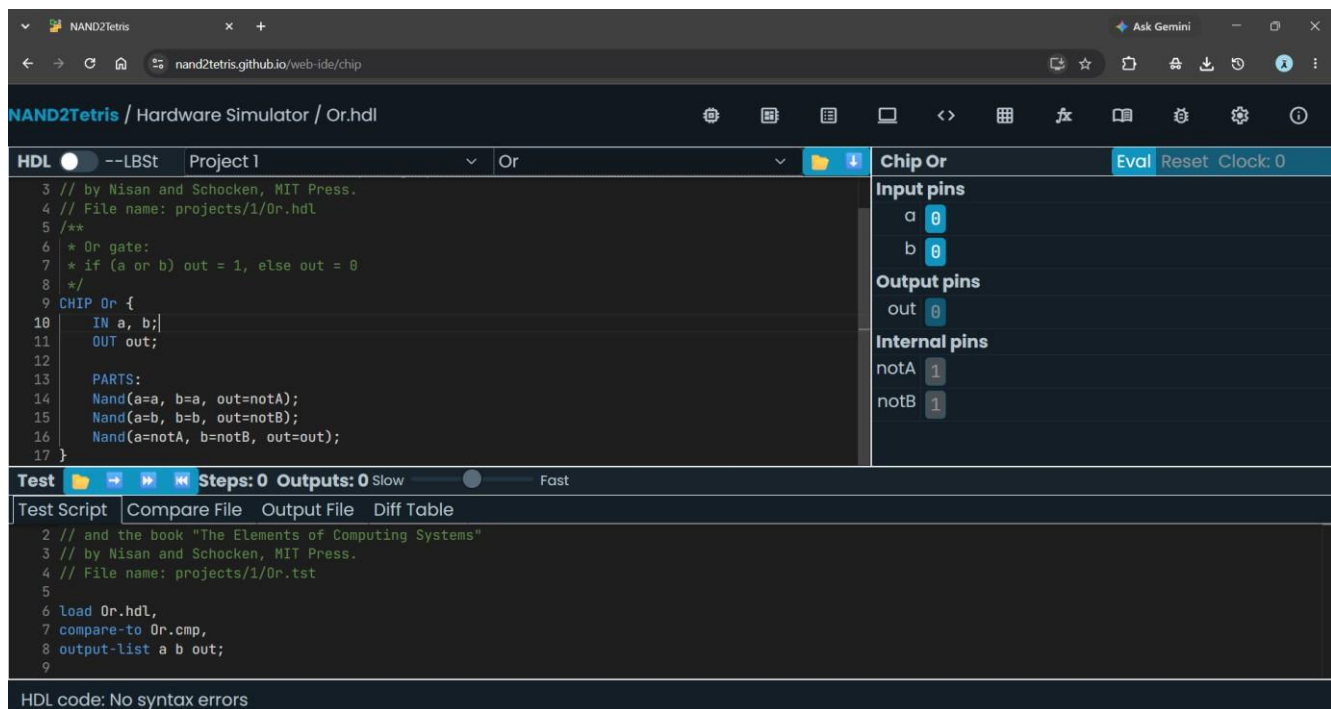


Figure 5. OR gate HDL implementation, Chip/Pins panel, and Or.tst test panel.